
Power Spectrum Estimation

Release 1.0.0

AE,SB,BB

Jun 01, 2026

Contents:

1	psestimation module	1
---	---------------------	---

PSESTIMATION MODULE

Quantity	Mathematical symbol	Codespace name
Centered frequency	ν_c	nuc
Separation between two frequency channels	$\Delta\nu_c$	dnuc
Number of frequency separations used to estimate cylindrical power spectrum	N_E	NE
Comoving distance at ν_c	r	r
Derivative of comoving distance w.r.t. frequency ν , evaluated at ν_c	$r' = \left. \frac{dr}{d\nu} \right _{\nu_c}$	rp
Slope of Wedge boundary line	$\frac{[k_{\parallel}]_H}{k_{\perp}} = \frac{r}{r'\nu_c}$	fac
Prefactor in Fourier Transform (<i>volume factor</i>)	$r^2 r' \Delta\nu_c (N_E - 1)$	vfac
Component of $\vec{k}$ perpendicular to line of sight	$k_{\perp}$	kper
Component of $\vec{k}$ parallel to line of sight	$k_{\parallel}$	kpara
Multi-angular power spectrum (MAPS)	$C_{\ell}(\Delta\nu)$	c1
Cylindrical power spectrum	$P(k_{\perp}, k_{\parallel})$	pk
Standard deviation of $P_N(k_{\perp}, k_{\parallel})$	$\delta P_N(k_{\perp}, k_{\parallel})$	dpkn
Mean of variable X	μ_{Est}	mu_est
Standard deviation of variable X	σ_{Est}	sigma_est
Scaled std of $P_N(k_{\perp}, k_{\parallel})$	$\delta P_N^{\text{True}}(k_{\perp}, k_{\parallel}) = \sigma_{\text{Est}} \delta P_N(k_{\perp}, k_{\parallel})$	dpk
Number of logarithmic bins	N_{Bin}	NBin
Binned k values	k	kk/keff
Binned power spectrum	$P(k)$	ppk
Error in estimating binned power spectrum	$\delta P(k)$	dppk
Dimensionless power spectrum	$\Delta^2(k)$	dk2
2σ uncertainty of $\Delta^2(k)$	$2\sigma(k)$	dpk2
Signal to noise ratio	$\text{SNR} = \Delta^2(k)/\sigma(k)$	snr
Upper limits of power spectrum	$\Delta_{\text{UL}}^2(k)$	ul

Table 1: Namespace conversions

`psestimation.build_essential(nuc, dnuc, NE, lval, model='Planck18')`

For a given set of values $\nu_c, \Delta\nu_c, N_E, \ell$ and appropriate cosmological model either **Planck18** or **FlatLambdaCDM**, it calculates comoving distance r at ν_c and its frequency derivative r' . It also calculates the slope of wedge boundary line, corresponding volume factor, and components of $\vec{k}$: $k_\perp$ and $k_\parallel$ respectively.

Parameters

- `nuc (float)` – Central frequency ν_c in units of MHz.
- `dnuc (float)` – Separation between two frequency channels $\Delta\nu_c$ in units of MHz.
- `NE (int)` – Number of frequency separations used in the analysis N_E .
- `lval (np.ndarray)` – Angular multipole ℓ values.
- `model (str {'Planck18', 'FlatLambdaCDM'}, optional)` – The name of cosmological model used to calculate **comoving distance** r . By default **Planck18** is used.

Returns

- `r (float)` – Comoving distance r at ν_c in units of Mpc.
- `rp (float)` – Derivative of comoving distance r w.r.t. frequency ν , evaluated at ν_c , $r' = \left. \frac{dr}{d\nu} \right|_{\nu_c} \approx \frac{c}{H(z)} \times \frac{(1+z)^2}{\nu_e=1420 \text{ MHz}}$ in units of Mpc/MHz.
- `fac (float)` – Slope of Wedge boundary line $[k_\parallel]_H = [r/r'\nu_c] k_\perp$.
- `vfac (float)` – Factor (**volume factor**) in Fourier Transform, $r^2 r' \Delta\nu_c (N_E - 1)$.
- `kper (np.ndarray)` – Component of $\vec{k}$, perpendicular to line of sight, $k_\perp = \ell/r$.
- `kpara (np.ndarray)` – Component of $\vec{k}$, parallel to line of sight, with :

$$k_{\parallel m} = \frac{\pi m}{r' \Delta\nu_c (N_E - 1)}, \quad m = 0, 1, \dots, N_E - 1.$$

Examples

```

>>> import numpy as np
>>> import myutils.psfuns.psestimation as pe
Imported myutils
Imported psfuncs,   Import Time : 01/06/26 | 04:05:54 PM
>>> nuc  = 154.255 # observing frequency in MHz
>>> dnuc = 0.04    # freq separation in MHz
>>> NE   = 668     # no of channels used to estimate PS
>>> # ell values array
>>> fname = '/home/cts23ph/git_package_myutils/tge/bin_info_1000007200.npz'
>>> lval  = (np.load(fname)['lval']).astype('int')
>>> r, rp, fac, vfac, kper, kpara = pe.build_essential(nuc, dnuc, NE, lval)
-----

Cosmological Model: Planck18
Redshift           : z  = 8.21
Comoving distance : r  = 9191.20 Mpc
rprime            : rp = 16.93 Mpc/MHz
Sep used          : NE = 668
-----

Wedge boundary slope (fac) : 3.519
DCT factor (vfac)         : 3.816e+10
kper.shape                : (20,)
kpara.shape               : (668,)
-----

```

```

>>> r, rp, fac, vfac, kper, kpara = pe.build_essential(nuc, dnuc, NE, lval,
...                                                    model = 'FlatLambdaCDM')
-----

Cosmological Model: FlatLambdaCDM
Redshift           : z  = 8.21
Comoving distance : r  = 9209.37 Mpc
rprime            : rp = 16.99 Mpc/MHz
Sep used          : NE = 668
-----

Wedge boundary slope (fac) : 3.514
DCT factor (vfac)         : 3.844e+10
kper.shape                : (20,)
kpara.shape               : (668,)
-----

>>> # kper  shape (len(lval),)
>>> # kpara shape (NE,)

```

Notes

For calculating comoving distance r , we use **Flat** Λ CDM cosmological model. The cosmological parameters are from either from [Planck 2018 results. VI. Cosmological parameters](#) or **FlatLambdaCDM**. The [Planck 18 Astropy](#) , [FlatLambdaCDM](#) and module have been used here.

1. For **Planck18** we have :

Quantity	Mathematical symbol	Value
Hubble Constant at present	H_0	67.66 km/s/Mpc
Density parameter of matter	Ω_{m0}	0.30966
Density parameter of baryonic matter	Ω_{b0}	0.04897
CMB temperature at present	$T_{\text{CMB}0}$	2.7255 K
Effective number of neutino species	N_{eff}	3.046
Mass of neutrinos	m_ν	[0, 0, 0.06] eV

Table 2: Cosmological parameters taken from [Planck 18](#) used by [Astropy module](#).

2. For **FlatLambdaCDM** model we have considered that there is no curvature, $\Omega_{k0} = 0$. In this, we have assumed that **matter and dark energy** are the only constituents of the Universe. Here we have taken the corresponding desity parameters $\Omega_{m0} = 0.30966$, $\Omega_{\Lambda0} = 0.69034$, $\Omega_{\text{total}0} = \Omega_{m0} + \Omega_{\Lambda0} = 1$. And the present value of **Hubble constant** is $H_0 = 67.66$ km/s/Mpc.

`psestimation.calc_A(na, nb)`

Parameters

- `na (int)` – Non negative integer greater than 2.
- `nb (int)` – Non negative integer greater than 2.

Returns

A – A 2D matrix of shape $(na \times nb)$, with

$$A_{\rho\lambda} = B_{\rho\lambda} \cos\left(\frac{\pi\rho\lambda}{nb-1}\right)$$

Where $\rho = 0, 1, \dots, (na-1)$, $\lambda = 0, 1, \dots, (nb-1)$

$$B_{\rho\lambda} = \begin{cases} 0.5 & \text{if } \lambda = 0 \text{ or } (nb-1) \\ 1.0 & \text{otherwise} \end{cases}$$

Return type

`np.ndarray`

Examples

```
>>> print(pe.calc_A(2,2))
[[ 0.5  0.5]
 [ 0.5 -0.5]]
>>> print(np.round(pe.calc_A(3,3),3))
[[ 0.5  1.   0.5]
 [ 0.5  0.  -0.5]
 [ 0.5 -1.   0.5]]
>>> print(np.round(pe.calc_A(4,4),3))
[[ 0.5  1.   1.   0.5]
 [ 0.5  0.5 -0.5 -0.5]
 [ 0.5 -0.5 -0.5  0.5]
 [ 0.5 -1.   1.  -0.5]]
>>> print(np.round(pe.calc_A(2,4),3))
[[ 0.5  1.   1.   0.5]
 [ 0.5  0.5 -0.5 -0.5]]
```

`psestimation.window(N)`

Returns a 4 term **Blackman-Nuttall** window function values.

$$F(a) = c_0 - c_1 \cos\left(\frac{2\pi a}{2N-2}\right) + c_2 \cos\left(\frac{4\pi a}{2N-2}\right) - c_3 \cos\left(\frac{6\pi a}{2N-2}\right) \quad a = 0, 1, \dots, 2N-3$$

Where

$$c_0 = 0.3635819, c_1 = 0.4891775, c_2 = 0.1365995, c_3 = 0.0106411$$

It has following properties :

$$F[(N-1) + a] = F[(N-1) - a]$$

and its also a periodic function with period $2N-2$, i.e.

$$F[a + \gamma(2N-2)] = F(a), \quad \text{with } \gamma = \pm 1, \pm 2, \dots$$

$a = 0, 1, \dots, 2N-3$ corresponds to frequency separation values $-(N-1)\Delta\nu_c, -(N-2)\Delta\nu_c, \dots, (N-2)\Delta\nu_c$. Precisely ,

$$\Delta\nu_a = (-N + a + 1)\Delta\nu_c$$

And this function returns the window function $\mathcal{W}_{\text{BN}}(\Delta\nu_a)$ with

$$a = N-1, N, \dots, 2N-3, 2N-2$$

Parameters

N (*int*) – Number of frequency separations used (must be $N \geq 2$) to estimate the cylindrical power spectrum $P(k_\perp, k_\parallel)$.

Returns

window_value – Blackmann-Nuttall window function array $\mathcal{W}_{\text{BN}}(\Delta\nu_n)$ $\left[\Delta\nu_n = n\Delta\nu_c, n = 0, 1, \dots, N-1\right]$ of length N , normalized to $\Delta\nu = 0$. The elements corresponds to $\Delta\nu_0 = 0, \Delta\nu_1 = \Delta\nu_c, \Delta\nu_2 = 2\Delta\nu_c, \dots, \Delta\nu_{N-1} = (N-1)\Delta\nu_c$.

Return type

`np.ndarray`

Examples

```
>>> np.set_printoptions(precision = 4, suppress = True)
>>> w = pe.window(N = 5) # Blackman Nuttall window of length 5.
>>> print(f"w = {w}")
w = [1.      0.702  0.227  0.0252 0.0004]
>>> w = pe.window(N = 7) # Blackman Nuttall window of length 7.
>>> print(f"w = {w}")
w = [1.      0.8555 0.5292 0.227  0.0613 0.0082 0.0004]
```

Notes

While estimating the cylindrical power spectrum $P(k_{\perp}, k_{\parallel})$ from MAPS $C_{\ell}(\Delta\nu)$, the assumption is that MAPS is periodic with period $(2N - 2)\Delta\nu_c$. But in reality it is not. So at the band edges it suffers a discontinuity which creates **artefacts** in the estimated **PS**. So thereby it is multiplied by window function $\mathcal{W}_{BN}(\Delta\nu)$, effectively making the product $\mathcal{W}_{BN}(\Delta\nu)C_{\ell}(\Delta\nu)$ periodic.

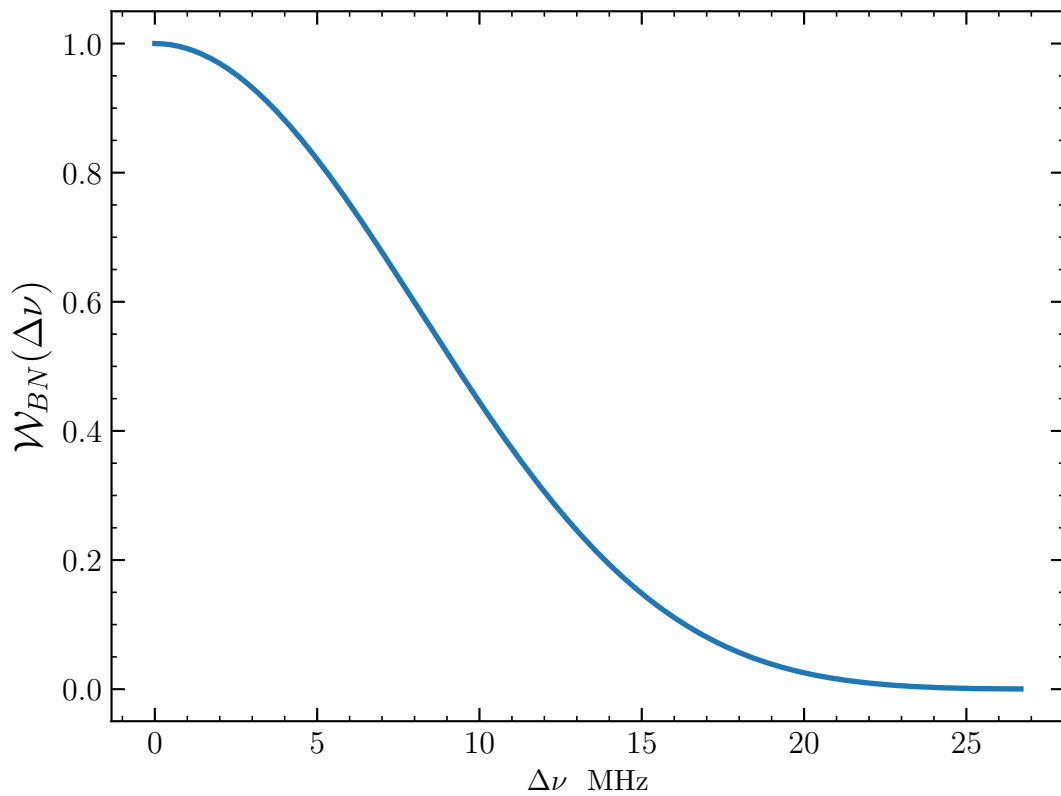


Fig. 1: Blackman-Nuttall window function for $N_E = 668$ with $\Delta\nu_c = 0.04$ MHz.

`psestimation.func_pk(cl, w, covi, vfac)`

For each annular bin in uv plane with effective multipole value ℓ_a , we have :

$$\begin{aligned}\mathbf{P}_a &= (\mathbb{A}^\dagger \mathbf{N}_a^{-1} \mathbb{A})^{-1} \mathbb{A}^\dagger \mathbf{N}_a^{-1} \times [\mathcal{W} \mathbf{C}_{\ell_a}] \\ &= \mathbb{M} \times [\mathcal{W}_{\text{BN}} \mathbf{C}_{\ell_a}]\end{aligned}$$

Where $\mathbb{A}$ matrix is constructed using **calc_A** function, $\mathcal{W}_{\text{BN}}$ is the window function. $\mathbf{P}_a$ is the cylindrical power spectrum value for fixed value of $k_{\perp a} = \ell_a/r$. In marix notation we can write :

$$\begin{bmatrix} P(k_{\perp a}, k_{\parallel m=0}) \\ P(k_{\perp a}, k_{\parallel m=1}) \\ \vdots \\ P(k_{\perp a}, k_{\parallel m=N_E-1}) \end{bmatrix} = \begin{bmatrix} \mathbb{M}_{00} & \mathbb{M}_{01} & \cdots & \mathbb{M}_{0,N_E-1} \\ \mathbb{M}_{10} & \mathbb{M}_{11} & \cdots & \mathbb{M}_{1,N_E-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbb{M}_{N_E-1,0} & \mathbb{M}_{N_E-1,1} & \cdots & \mathbb{M}_{N_E-1,N_E-1} \end{bmatrix} \times \begin{bmatrix} \mathcal{W}_{\text{BN}}(\Delta\nu_{n=0}) \times C_{\ell_a}(\Delta\nu_{n=0}) \\ \mathcal{W}_{\text{BN}}(\Delta\nu_{n=1}) \times C_{\ell_a}(\Delta\nu_{n=1}) \\ \vdots \\ \mathcal{W}_{\text{BN}}(\Delta\nu_{n=N_E-1}) \times C_{\ell_a}(\Delta\nu_{n=N_E-1}) \end{bmatrix}$$

Parameters

- **cl** (*np.ndarray*) – Multi-angular power spectrum $C_\ell(\Delta\nu)$ (MAPS) array. Shape $(\dots, \text{len}(\text{lval}), NE)$.
- **w** (*np.ndarray*) – The window function normalized to $\Delta\nu = 0$, which is used to avoid discontinuity at the band edges. Shape $(NE,)$.
- **covi** (*np.ndarray*) – Inverse of diagonal elements of noise covariance matrix $\mathbf{N}$ estimated from multiple noise only simulations. Shape $(\dots, \text{len}(\text{lval}), NE)$.
- **vfac** (*int or float*) – The volume factor $r^2 r' \Delta\nu_c (NE - 1)$ sitting in the FT of MAPS.

Returns

pk – Cylindrical power spectrum $P(k_{\perp}, k_{\parallel})$ array using MLE (**maximum likelihood estimator**). Shape $(\dots, \text{len}(\text{lval}), NE)$.

Return type

`np.ndarray`

Examples

```

>>> index = 7200
>>> cl = np.load(f'/home/cts23ph/git_package_myutils/clfuncs/cldnur_{index}.npy')[...
↪, :NE]
>>> cln = np.load(f'/home/cts23ph/git_package_myutils/clfuncs/cldnur_noise_{index}.
↪npz')[..., :NE]
>>> dcln = np.std(cln, axis = 0)
>>> covi = 1/dcln**2
>>> # window for Fourier transform
>>> w = pe.window(NE)
>>> # MLE cylindrical PS P(kper, kpara)
>>> pk = pe.func_pk(cl, w, covi, vfac)
>>> print(f'{cl.shape      = }\n
...      \n{covi.shape    = }\n
...      \n{w.shape       = }\n
...      \n{pk.shape      = }')
cl.shape      = (20, 668)
covi.shape     = (20, 668)
w.shape        = (668,)
pk.shape       = (20, 668)

```

Notes

Here we have assumed that the noise covariance matrix $\mathbf{N}$ is diagonal along frequency separation axis $\Delta\nu$ and for each annular bin ℓ_a bin we have :

$$(\mathbf{N}_a)_{mn} = \delta_{mn} [\delta C_{\ell_a}(\Delta\nu_n)]_{\text{noise}}^2$$

So each element of covi (the last axis) contains the value $1/[\delta C_{\ell_a}(\Delta\nu_n)]_{\text{noise}}^2$.

The explicit form of the matrix $\mathbf{N}_a$ is given as :

$$\begin{bmatrix} [\delta C_{\ell_a}(\Delta\nu_{n=0})]_{\text{noise}}^2 & 0 & \cdots & 0 \\ 0 & [\delta C_{\ell_a}(\Delta\nu_{n=1})]_{\text{noise}}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & [\delta C_{\ell_a}(\Delta\nu_{n=N_E-1})]_{\text{noise}}^2 \end{bmatrix}$$

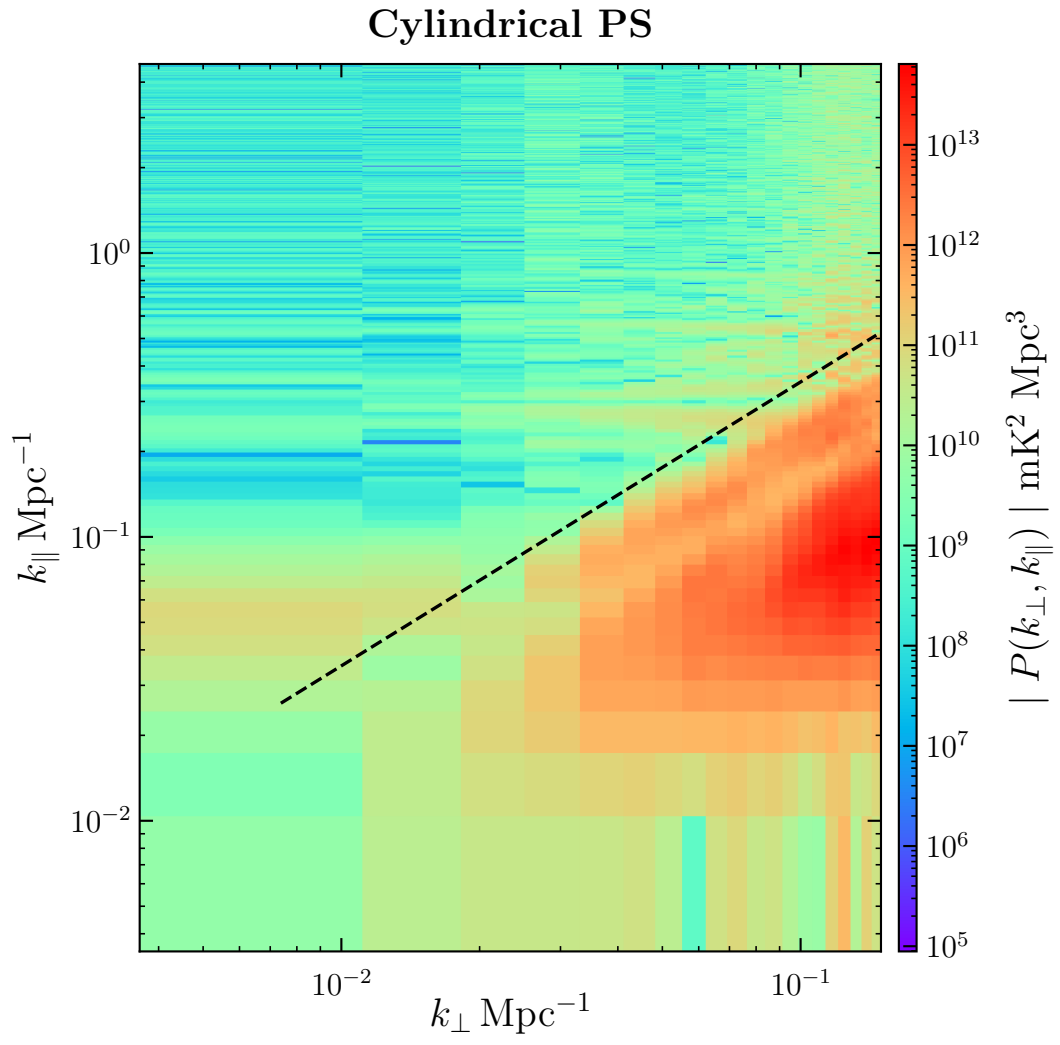


Fig. 2: Cylindrical Power Spectrum $P(k_{\perp}, k_{\parallel})$.

psestimation.X(pk, dpkn, flag_mask)

Parameters

- **pk** (*np.ndarray*) – Cylindrical power spectrum $P(k_{\perp}, k_{\parallel})$ array (**2D or higher dimensional array**), Must be of shape $[\dots, \text{len}(k_{\text{per}}), \text{len}(k_{\text{para}})]$.
- **dpkn** (*np.ndarray*) – Standard deviation of cylindrical power spectrum $\delta P_N(k_{\perp}, k_{\parallel}) = \sqrt{\langle [P_N(k)]^2 \rangle - \langle P_N(k) \rangle^2}$, estimated from noise only simulations. Where $\langle \dots \rangle$ denotes average over realizations and $P_N(k_{\perp}, k_{\parallel})$ is the cylindrical power spectrum obtained using noise only simulations. **Must have same shape as pk.**
- **flag_mask** (*np.ndarray*) – A **2D** boolean mask of shape $[\text{len}(k_{\text{per}}), \text{len}(k_{\text{para}})]$. If the element value is **1** then that $(k_{\perp}, k_{\parallel})$ **mode is used**, if **0** then **rejected**. Should contain values **0 and 1 only**.

Returns

- **X** (*np.ndarray*) – The values of variable $X = \frac{P(k_{\perp}, k_{\parallel})}{\delta P_N(k_{\perp}, k_{\parallel})}$ (**flattened array along last two dimensions**) corresponding to those $(k_{\perp}, k_{\parallel})$ modes defined via **flag_mask** boolean array. Shape $[\dots, \text{modes}]$. Where *modes* is the number of elements containing **1** in **flag_mask** array.
- **mu_est** (*float or np.ndarray*) – Mean value of X (μ_{Est}), estimated using $(k_{\perp}, k_{\parallel})$ modes defined in **flag_mask** boolean array. Shape $\text{pk.shape}[: -2]$. **last two axes are removed.**
- **sigma_est** (*float or np.ndarray*) – Standard deviation of X (σ_{Est}), estimated using $(k_{\perp}, k_{\parallel})$ modes defined in **flag_mask** boolean array. Same shape as **mu_est**.

Examples

```
>>> # Noise Power Spectrum
>>> # MLE cylindrical PS P(kper, kpara)          # noise
>>> pkn    = pe.func_pk(cln, w, covi, vfac)      # noise ps
>>> dpkn   = np.std(pkn, axis = 0)              # std noise of ps
>>> dpkn *= (20)**2                             # scale the noise accordingly
>>> # flag mask array
>>> fm = np.load('/home/cts23ph/git_package_myutils/psfuncs/flag_mask.npy')
>>> print(f'{pkn.shape          = }\n\
...      \n{dpkn.shape        = }\n\
...      \n{fm.shape          = }')
pkn.shape          = (50, 20, 668)
dpkn.shape         = (20, 668)
fm.shape           = (20, 668)
>>> X, mu, sigma = pe.X(pk, dpkn, fm) # X statistics
```

(continues on next page)

(continued from previous page)

```
>>> np.set_printoptions(precision = 3, suppress = True)
>>> print(f'X.shape = {X.shape}\n\nmu = {mu}\n\nsigma = {sigma}')
X.shape = (503,)
mu = 0.2797964155866535
sigma = 1.2258385055783663
```

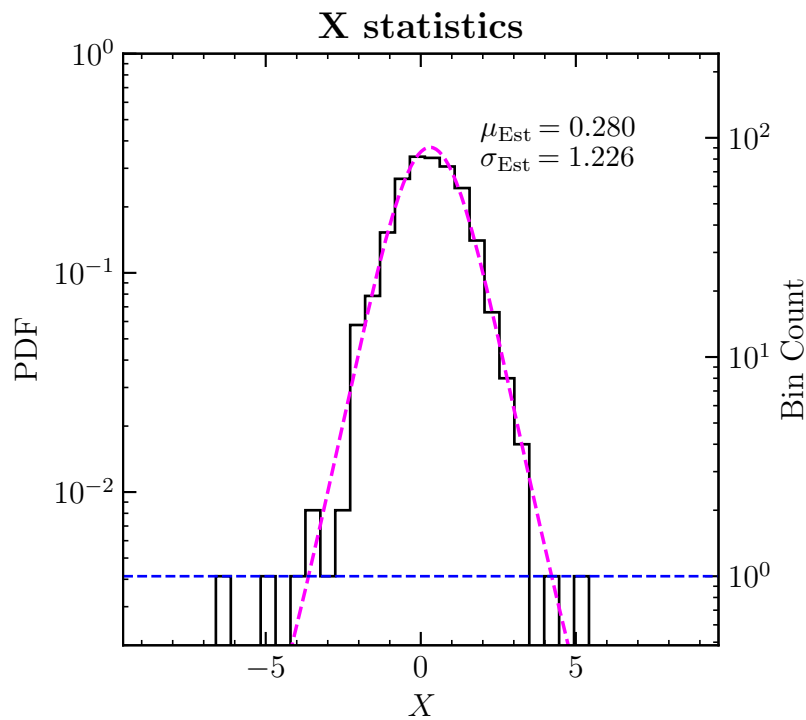


Fig. 3: X statistics. The magenta colored line shows student t fit.

Notes

Here is how you can make flag mask array.

```
>> flag_mask = np.zeros((kper.size,kpara.size), dtype = 'int')
>> print(flag_mask.shape)
>> ks = np.array([0.135, 0.228, 0.36, 0.5, 0.72, 0.8, 0.92, 1.09, 1.17, 1.39])
>> ksv = np.array([0,0,1,2,2,2])
>> for ii in range(len(ksv)):
>>     for jj in range(ksv[ii],len(ksv)-1):
>>         mask = (kpara>=ks[2*jj])*(kpara<=ks[2*jj+1])
>>         flag_mask[ii,mask] = 1
>> np.save('flag_mask.npy',flag_mask) # save the array
```

Where the modes used is tabulated below :

$k_{\perp} \text{ Mpc}^{-1}$	$k_{\parallel} \text{ Mpc}^{-1}$				
0.007	0.135 – 0.228	0.360 – 0.499	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399
0.015	0.135 – 0.228	0.360 – 0.499	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399
0.022	–	0.360 – 0.499	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399
0.029	–	–	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399
0.038	–	–	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399
0.045	–	–	0.720 – 0.797	0.921 – 1.095	1.171 – 1.399

Table 3: $k_{\parallel}$ ranges corresponding to different $k_{\perp}$ values that have been used to estimate binned PS and X statistics.

`psestimation.binned_pk(kper, kpara, pk, dpk, NBin, flag_mask)`

Using the mode information contained in **flag_mask**, first it computes all the $k (= \sqrt{k_{\perp}^2 + k_{\parallel}^2})$ values available and bins the $(k_{\perp}, k_{\parallel})$ space into $NBin$ no of **logarithmic bins**. And for each bin k_i , it computes binned power spectrum values $P(k_i)$ and the corresponding error $\delta P(k_i)$ using the **minimum variance estimator**. See notes for more details.

$$P(k_i) = \frac{\sum_{(k_{\perp}, k_{\parallel}) \in i} \mathbf{w}(k_{\perp}, k_{\parallel}) P(k_{\perp}, k_{\parallel})}{\sum_{(k_{\perp}, k_{\parallel}) \in i} \mathbf{w}(k_{\perp}, k_{\parallel})}$$

$$\delta P(k_i) = \left[\sum_{(k_{\perp}, k_{\parallel}) \in i} \mathbf{w}(k_{\perp}, k_{\parallel}) \right]^{-1/2}$$

$$k_i = \exp \left[\frac{\sum_{(k_{\perp}, k_{\parallel}) \in i} \mathbf{w}(k_{\perp}, k_{\parallel}) \ln(k)}{\sum_{(k_{\perp}, k_{\parallel}) \in i} \mathbf{w}(k_{\perp}, k_{\parallel})} \right];$$

Where $\mathbf{w}(k_{\perp}, k_{\parallel})$ is the weight assigned to that particular $(k_{\perp}, k_{\parallel})$ mode and by $(\sum_{k_{\perp}, k_{\parallel}}) \in i$, we denote, the sum is being performed using all the $(k_{\perp}, k_{\parallel})$ modes which lies in i^{th} bin.

Parameters

- **kper** (*np.ndarray*) – **1d array** containing the values of wave-vector $\vec{k}$ **perpendicular** to the line of sight (**LOS**). Denoted by sign $k_{\perp}$.
- **kpara** (*np.ndarray*) – **1d array** containing the values of wave-vector $\vec{k}$ **along** to the line of sight (**LOS**). Denoted by sign $k_{\parallel}$.
- **pk** (*np.ndarray*) – Cylindrical power spectrum $P(k_{\perp}, k_{\parallel})$ array (**2D or higher dimensional array**), Must be of shape $[\dots, \text{len}(kper), \text{len}(kpara)]$.
- **dpk** (*np.ndarray*) – Standard deviation of cylindrical noised power spectrum (scaled) $\delta P_N^{True}(k_{\perp}, k_{\parallel}) = \sigma_{\text{Est}} \delta P_N(k_{\perp}, k_{\parallel})$ array (**2D or higher dimensional array**), Must be of shape $[\dots, \text{len}(kper), \text{len}(kpara)]$, same shape as **pk**.
- **NBin** (*int*) – No of bins the whole $(k_{\perp}, k_{\parallel})$ space has been divided into using only the modes defined in **flag_mask** mode info.
- **flag_mask** (*np.ndarray*) – A **2D** boolean mask of shape $[\text{len}(kper), \text{len}(kpara)]$. If the element value is **1** then that $(k_{\perp}, k_{\parallel})$ **mode is used**, if **0** then **rejected**. Should contain values **0 and 1 only**.

Returns

- **keff** (*np.ndarray*) – The binned k values (**1d array**). Shape ($nzNBin,$).
- **ppk** (*np.ndarray*) – Binned power spectrum $P(k)$ values (**Spherical PS**), (**n-dimensional array**). Shape $:(\dots, nzNBin)$.
- **dppk** (*np.ndarray*) – 1σ error in estimating spherical ps, $\delta P(k)$ values, (**n-dimensional array**). Shape $:(\dots, nzNBin)$.

Examples

```
>>> NBin = 5
>>> kk, ppk, dppk = pe.binned_pk(kper, kpara, pk, sigma*dpkn,
...                               NBin, fm)
Bin count   : [ 26   3  60  96 318]
>>> # kk   = binned k values
>>> # ppk  = binned P(k) values
>>> # dppk = 1 sigma error in estimating binned P(k) values
>>> print(f'{kk.shape   = }\n
...       \n{ppk.shape  = }\n
...       \n{dppk.shape = }')
kk.shape    = (5,)
ppk.shape   = (5,)
dppk.shape  = (5,)
>>> NBin = 8
>>> kk, ppk, dppk = pe.binned_pk(kper, kpara, pk, sigma*dpkn,
...                               NBin, fm)
Bin count   : [ 15  11   0  45  18  72 150 192]
>>> # kk   = binned k values
>>> # ppk  = binned P(k) values
>>> # dppk = 1 sigma error in estimating binned P(k) values
>>> print(f'{kk.shape   = }\n
...       \n{ppk.shape  = }\n
...       \n{dppk.shape = }')
kk.shape    = (7,)
ppk.shape   = (7,)
dppk.shape  = (7,)
```


Notes

1. If no $(k_{\perp}, k_{\parallel})$ mode contributes to a bin, then we reject those bins in the end.
2. $nzNBin$ is the altered value of number of bins $NBin$. This happens as it might happen number of modes contributes is **zero**.
3. The binning is performed via the **minimum-variance estimator** where we have chosen the weights as :

$$\mathbf{w}(k_{\perp}, k_{\parallel}) = \frac{1}{[\delta P_N^{True}(k_{\perp}, k_{\parallel})]^2} = \frac{1}{[\sigma_{Est} \times \delta P_N(k_{\perp}, k_{\parallel})]^2}$$

`psestimation.func_dT(kk, ppk, dppk)`

Given the binned k values, binned power spectrum $P(k)$ and its 1σ error $\delta P(k)$, it gives the **dimensionless power spectrum** $\Delta^2(k)$ and its 2σ uncertainty, signal to noise ratio **SNR** and the corresponding **upper limit** $\Delta_{\text{UL}}^2(k)$.

Parameters

- **kk** (*np.ndarray*) – The binned k values (**1d array**). Shape $(\text{len}(kk),)$.
- **ppk** (*np.ndarray*) – Binned power spectrum $P(k)$ values, (**n-dimensional array**). Shape $(\dots, \text{len}(kk))$.
- **dppk** (*np.ndarray*) – 1σ error in estimating spherical ps, $\delta P(k)$ values, (**n-dimensional array**). Same shape as **ppk**.

Returns

- **dk2** (*np.ndarray*) – Dimensionless power spectrum $\Delta^2(k)$, (**n-dimensional array**). Same shape as **ppk**.
- **dpk2** (*np.ndarray*) – Statistical (2σ) error in estimating dimensionless power spectrum $\Delta^2(k)$, (**n-dimensional array**). Same shape as **ppk**.
- **snr** (*np.ndarray*) – Signal to noise ratio. (**n-dimensional array**). Same shape as **ppk**.
- **ul** (*np.ndarray*) – Upper limit of dimensionless power spectrum $\Delta_{\text{UL}}^2(k)$, (**n-dimensional array**). Same shape as **ppk**.

They are defined as follows :

$$\Delta^2(k) = \frac{k^3 P(k)}{2\pi^2}$$

$$\sigma(k) = \frac{k^3 \delta P(k)}{2\pi^2}$$

$$\text{SNR} = \Delta^2(k) / \sigma(k) = P(k) / \delta P(k)$$

$$\Delta_{\text{UL}}^2(k) = \begin{cases} \Delta^2(k) + 2\sigma(k) & \text{if } \Delta^2(k) > 0 \\ 2\sigma(k) & \text{otherwise} \end{cases}$$

- .. raw:: latex – newpage

Examples

```
>>> dk2, dpk2, snr, ul = pe.func_dT(kk, ppk, dppk)
>>> print(f'{dk2.shape = }\n\
...      \n{dpk2.shape = }\n\
...      \n{snr.shape = }\n\
...      \n{ul.shape = }')
dk2.shape = (7,)
dpk2.shape = (7,)
snr.shape = (7,)
ul.shape = (7,)
```

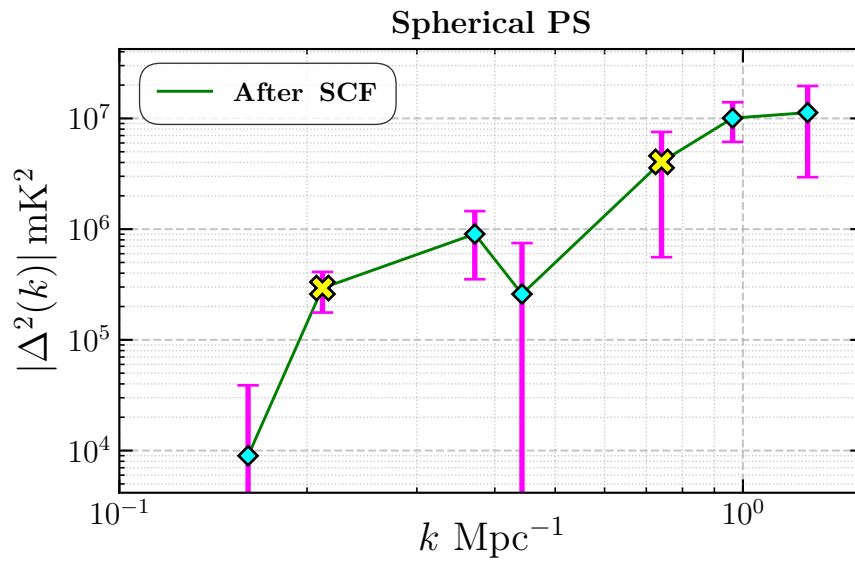


Fig. 4: Spherical Power Spectrum $P(k)$.